zkPHIRE: A Programmable Accelerator for ZKPs

Source: zkPHIRE: A Programmable Accelerator for ZKPs over HIgh-degRee, Expressive Gates

Team members: Sammy Suliman, Jui-Teng Huang, Ashesh Kaji
Topic: We want to develop a hardware accelerator for one round (or full loop) of the Sum-check protocol over a finite field.

At a high level, each round does:

1. Fix previous variables
2. Evaluate a polynomial over many points
3. Sum results to produce a univariate polynomial
4. Output coefficients → next round

Motivation: The Sum-Check protocol is well-suited for hardware acceleration because it relies on many parallelizable multiplications. We need to evaluate a function on many inputs**,** then sum them– all independently.

SumCheck repeatedly does **"sum over many values"** operations.

Hardware implements this as a **reduction tree**:

```
a1   a2   a3   a4   a5   a6   a7   a8

\  /    \  /    \  /    \  /

 s1       s2       s3       s4

   \     /          \     /

      t1                t2

         \           /

              total
```

This structure:

- Has low latency (logarithmic depth)
- Is easy to pipeline
- Is extremely efficient in hardware

SumCheck is **round-based**:

- Round 1 → compute g1g1
- Round 2 → compute g2g2
- …

Each round:

- Takes inputs
- Produces a small output polynomial
- Feeds into the next round

This is ideal for **pipelining**:

- While one round is finishing, the next can start
- Data flows continuously through the hardware

So hardware can:

- Compute thousands of f(·)f(\cdot)f(·) evaluations at the same time
- Reduce them with parallel adders (tree reduction)
- Combine them together in a pipeline

<u>SumCheck Protocol:</u>

SumCheck is a **protocol to verify that a sum over a big polynomial is correct** without computing the whole sum yourself.

Imagine you have a polynomial f(x_1, x_2, ..., x_n) and you claim that:

$$s_i(X_i) = \sum_{x_{i+1} \in \{0,1\}} \cdots \sum_{X_\mu \in \{0,1\}} f(r_1, \ldots, r_{i-1}, X_i, \ldots, X_\mu)$$

You (the prover) want to convince someone else (the verifier) that SSS is correct **without actually sending all 2^n evaluations** (which would be huge for large n).

SumCheck lets the verifier check this **efficiently** by interacting with the prover.

How it works (step by step)

1. **Start with the first variable x1x_1x1:**

- The prover computes a new polynomial g1(x1)=∑x2,...,xnf(x1,x2,...,xn)g_1(x_1) = \sum_{x_2,...,x_n} f(x_1, x_2,...,x_n)g1(x1)=∑x2,...,xnf(x1,x2,...,xn).
- This is a **polynomial in just x1x_1x1**.
- The prover sends g1g_1g1 to the verifier.

2. **Verifier checks a sum:**
   - Verifier picks a random value r1r_1r1 for x1x_1x1 and asks the prover to prove that g1(r1)=∑x2,...,xnf(r1,x2,...,xn)g_1(r_1) = \sum_{x_2,...,x_n} f(r_1, x_2,...,x_n)g1(r1)=∑x2,...,xnf(r1,x2,...,xn).
3. **Repeat for each variable:**
   - For x2x_2x2, the prover creates g2(x2)=∑x3,...,xnf(r1,x2,...,xn)g_2(x_2) = \sum_{x_3,...,x_n} f(r_1, x_2, ..., x_n)g2(x2)=∑x3,...,xnf(r1,x2,...,xn).
   - Verifier picks r2r_2r2 and asks for g2(r2)g_2(r_2)g2(r2).
   - Continue until all variables are “fixed” by random values.
4. **Final check:**
   - Eventually, you’re left with **a single number**, f(r1,r2,...,rn)f(r_1, r_2, ..., r_n)f(r1,r2,...,rn), which the verifier can compute directly.
   - If all checks pass, the sum claim is probably correct.

Key idea: verifier only does **linear work in n**, not exponential 2n2^n2n.

<u>Summary (Sammy’s Transcribed Notes)</u>:

Zero Knowledge Proofs

- Enable one party to convince another of a statement’s validity **without revealing anything else**
- High computational overhead → motivates **hardware acceleration**
- ZKP protocols rely on **SumCheck protocol** *(O(N))*
- Encode computations as **arithmetic circuits (AC)**

SumCheck

- Custom, high-degree gates
  - Example:
    f=a⋅b5+cf = a \cdot b^5 + cf=a⋅b5+c
- Reduces the number of gates needed

Challenges

- Computations vary with:
  - Each gate’s structure

    - Polynomial degree
- Cannot easily handle complex gates

Introducing zkPHIRE

- A novel **programmable SumCheck unit**
- Supports:
    - High-degree multilinear polynomials
    - Compatible gate types

Zero Knowledge Proofs (Properties / Prereqs)

- Verifier learns nothing beyond validity of the statement
- Verified quickly
- No back-and-forth communication

Multi-Scalar Multiplication

- Compute:
  S=∑siPiS = \sum s_i P_iS=∑siPi
  along an elliptic curve
- Computationally intensive

SumCheck

$$s_i(X_i) = \sum_{x_{i+1} \in \{0,1\}} \cdots \sum_{X_\mu \in \{0,1\}} f(r_1, \ldots, r_{i-1}, X_i, \ldots, X_\mu)$$

- Prover (P) wants to convince Verifier (V):
  f(X1,…,Xn)=Cf(X_1, \ldots, X_n) = Cf(X1,…,Xn)=C
  where inputs are **Boolean**

Iterative Definition

For 1≤i≤n1 \le i \le n1≤i≤n:

si=∑xi∈{0,1}∑f(r1,r2,…,ri−1,xi,Xi+1,…,Xn)s_i = \sum_{x_i \in \{0,1\}} \sum f(r_1, r_2, \ldots, r_{i-1}, x_i, X_{i+1}, \ldots, X_n)si=xi∈{0,1}∑∑f(r1,r2,…,ri−1,xi,Xi+1,…,Xn)

- r1,…,ri−1r_1, \ldots, r_{i-1}r1,…,ri−1 are **verifier's random challenges** from previous rounds

---

Multilinear Extension (MLE)

- Multilinear extension = polynomials with:
  - Degree ≤ 1 in each variable

Plonk

f=qLw1+qRw2+qMw1w2−qOw3+qCf = q_L w_1 + q_R w_2 + q_M w_1 w_2 - q_O w_3 + q_Cf=qLw1+qRw2+qMw1w2−qOw3+qC

- Encodes an operation
- Evaluated to compute fff

Encoding

X=(X1,X2,…)X = (X_1, X_2, \ldots)X=(X1,X2,…)

- Represents **binary-encoded gate index**
- Example:
  (1,1,0)→gate 3(1,1,0) \rightarrow \text{gate 3}(1,1,0)→gate 3
- Functions:
  - qL(X),w2(X),…q_L(X), w_2(X), \ldotsqL(X),w2(X),…

Alternative Explanation of SumCheck

gj(t)=∑F(r1,…,rj−1,t,Xj+1,…,Xn)g_j(t) = \sum F(r_1, \ldots, r_{j-1}, t, X_{j+1}, \ldots, X_n)gj(t)=∑F(r1,…,rj−1,t,Xj+1,…,Xn)

Ex: (Sum/Check)

n=2 (variables)

k=2 (polynomials)

$f_1(x_1, x_2) = 1 + 2x_1 + 3x_2 + 4x_1x_2$

$f_2(x_1, x_2) = 5 + 6x_1 + 7x_2 + 8x_1x_2$

$E(x) = f_1 - f_2$

$S = \sum_{(x1,x2} \in \{0,1,3\}) F(x_1, x_2)$

X = {10, 0}, (0, 2), (1, 0), (1, 1, 3)}

S = 5 + 33 + 48 + 260 = 346

⭐ $g_1(1) = \sum_{(x2} \in \{0,3\}) F(1, x_2)$ ⭐ WTS $\sum_{(1)} g(1) = S$

Let $x_2=0$, ... $f_1(1,0)=1+2t$, $f_2(1,0)=5+6t$

$\Rightarrow (1+2t)(5+6t)=5+16t+12t^2$

Let $x_2=1$: $f_1(1,1)=4+6t$, $f_2(1,1)=12+14t$

Now, $g_1(0)=f_1(0,0)+f_2(0,0)=53$

$g_1(1)=f_1(1,1)+f_2(1,1)=293$

S=53+293=346 ✓

Assuming $x_1=r_1$, what's the remaining partial sum over x_2?

New claim: $S_2 = \sum_{(x2} \in \{0,3\}) F(r_1, x_2) = g_1(r_1)$

$= f_2(r_1, 0) + f_2(r_1, 1) + f_2(r_1, 1) + f_2(r_1, 1)$

$= f_2(2, 0) - f_2(2, 0) + f_2(2, 1) - f_2(2, 1)$

$= (5+12)(1+4)+(1+4+3+8)(5+12+7+16)$

$= (17 \cdot 5) + (16 \cdot 40)$

$= 85 + 640$

$= 725$

Sumcheck of as opportunities for parallelism, but this voids opportunities for data reuse.

Instead of streaming all individuals, polynomials MLF ends simultaneously, we proceed one term at a time.

Hardware acceleration

$f_1(0, x_2, ...) \rightarrow f_1(1, x_2, ...)$

$f_1(1, x_2, ...) \rightarrow f_1(2, x_2, ...)$

$\rightarrow f_1(2, 0) - f_1(2, 0) + f_1(2, 1) - f_1(2, 1)$

$= (5+12)(1+4)+(1+4+3+8)(5+12+7+16)$

$= (17 \cdot 5) + (16 \cdot 40)$

$= 85 + 640$

$= 725$

ZKPHIRE advancements

- Implement product trees (parallel processing paths that independently compute and one $f_1(x)$, $f_2(x)$, $f_3(x)$)

- These modular multiplies can be reconfigured for other protocol steps (not just sumcheck)

Architecture

Input Stream → | Input Buffer |

↓ (AXI-Stream FIFO)

| Polynomial Eval | ← main compute

↓ (AXI-Stream FIFO)

| Accumulator | ← sum over domain

↓ (AXI-Stream FIFO)

| Coeff Extractor |

↓ (AXI-Stream FIFO)

| Controller (FSM) |

Finite field unit

Everything runs over a field F_p

Pick something hardware-friendly:

- 32-bit prime (easy FPGA)

Polynomial Evaluation Unit (PEU)

This is your **main datapath**.

What it does

Evaluates something like:

g(x1,...,xn)g(x_1, ..., x_n)g(x1,...,xn)

But in practice:

- many terms
- often multilinear

Hardware strategy

Use **Horner-like evaluation**:

Instead of:

ax2+bx+ca x^2 + b x + cax2+bx+c

Do:

y = a

y = y * x + b

y = y * x + c

Implementation

- Chain of **MAC units** (multiply-accumulate)
- Pipeline it:

Stage 1: multiply

Stage 2: add

Stage 3: reduce mod p

Parallelism

Instantiate multiple PEUs:

- 4, 8, 16 lanes (depending on FPGA)

Each evaluates a different input point.

Input generation (very important)

You don't want to store all inputs.

Instead:

> Generate evaluation points **on the fly**

Example:

For Boolean hypercube:

- inputs $\in \{0,1\}^n$

Use a **counter-based generator**:

- Treat index as bitmask
- Each bit = variable value

Benefit:

- Zero memory cost
- Perfect streaming

Accumulator (critical bottleneck)

This computes:

∑g(x)\sum g(x)∑g(x)

Design

- Tree of adders OR
- Running accumulator

Option A: Serial accumulator

- One adder
- Low area
- Slower

Option B: Parallel reduction tree

- Fast
- More LUT usage

👉 Start with **serial**, then optimize.

Coefficient extractor

In sumcheck, each round outputs a **univariate polynomial**:

h(t)=a0+a1t+a2t2+...h(t) = a_0 + a_1 t + a_2 t^2 + ...h(t)=a0+a1t+a2t2+...

Trick

Instead of symbolic math:

- Evaluate at **multiple points**
- Interpolate coefficients

Hardware approach:

1. Evaluate at:
    - t = 0
    - t = 1
2. Solve small system (cheap)

Controller (FSM)

Controls rounds:

IDLE

→ LOAD

→ EVALUATE (stream inputs)

→ ACCUMULATE

→ OUTPUT

→ NEXT ROUND

Responsibilities:

- Reset accumulators
- Trigger PEU pipelines
- Track iteration count

External CPU (Host-PS Interaction)

1. Physical connection

We need some way to send data and commands from the PC to the FPGA:

1. **USB** – simple, low-to-medium speed.
2. Send messages using **UART** protocol
3. Interface with hardware using a Python script and pynq library

Expose hardware registers/memory in PL:

- Use **AXI Lite / AXI DMA** interfaces to map registers or buffers that the PS can access.
- Example:
    - Register 0x00 → Start signal
    - Register 0x04 → Round number / config
    - DMA buffer → Polynomial data input/output

USB communication:

- PC talks to the PS (on FPGA) via **USB-UART bridge**.

- The PS acts as a bridge between PC software and PL hardware.

PYNQ on the PS:

- Load your **bitstream (.bit file)** and overlay in PYNQ.
- Map memory/registers from PL to Python-accessible objects.
- Provide Python functions to **write commands, send input, read results**.

Component	Role
PC (Host)	Generates input data (polynomial coefficients, circuit description), sends control commands, receives results, handles randomness for ZKP.
PS (Processing System on FPGA)	Acts as a bridge: receives PC commands over USB, writes data to PL (hardware accelerator), triggers computation, reads results from PL, sends results back to PC.
PL (FPGA Fabric / Accelerator)	Performs the heavy-lifting computations (SumCheck polynomial sums, reductions, etc.).

Think of these as the **"conversation"**:

1. **Configuration / Initialization**
    - Send metadata about the polynomial:
        - Number of variables, polynomial degree, number of terms
        - Memory addresses for input/output buffers
    - Example: `CONFIG {num_vars=10, degree=3, buffer_addr=0x1000}`
2. **Input Data Transfer**
    - Send the actual **polynomial coefficients** (and any other data needed by SumCheck).
    - For large data, use **DMA** to transfer blocks of coefficients efficiently.
3. **Control Commands**
    - Start computation: e.g., `START_SUMCHECK`
    - Optional: Pause, reset, or stop signals

    - Could be simple 1-bit register writes via AXI-Lite.
4. **Verifier Random Challenges**
    - In SumCheck, the verifier picks random values per round.
    - Host sends the random challenge to PS → PL each round:
        - `CHALLENGE {r1}`
        - `CHALLENGE {r2}` …
5. **Status Queries**
    - Host can poll a **status register** to see if the accelerator is busy or finished.
    - Example: `STATUS?` → returns `BUSY` or `DONE`
6. **Retrieve Results**
    - Once PL finishes computation, host reads results:
        - SumCheck partial sums
        - Final polynomial evaluation
    - Example: `READ_RESULT` → returns array of results

---

3. PC-side workflow (sending/receiving messages)

1. **Send initialization/config command**
2. **Transfer input data (coefficients, circuit description)**
3. **Trigger computation (START command)**
4. **Send random challenges for each round**
5. **Poll or wait for done signal**
6. **Read results from PL via PS**
7. **Repeat next round if needed**

```
┌──────────────────────┐
│     PC Host          │
│ (Python / PYNQ)      │
└──────────┬───────────┘
           │ USB
           ▼
┌──────────────────────┐
│  PS (ARM CPU)        │
│ on FPGA / Zynq       │
```

```
└──────────┬──────────┘
           │ AXI / DMA / Registers
           ▼
┌──────────────────────┐
│ PL (FPGA Logic)      │
│ SumCheck Pipeline    │
└──────────────────────┘
```

9) Memory design

Keep it minimal:

On-chip:

- small buffers for:
  - coefficients
  - partial sums

Avoid:

- large BRAM usage

---

10) Dataflow (this is key)

You want:

Fully streaming pipeline

Input → PEU → Accumulator → Output

↑

pipelined

Goal:

- 1 evaluation per cycle (after pipeline fill)

---

11) Performance target (realistic)

For a good FPGA design:

- Throughput:
  ~1–10 million evaluations/sec
- Latency:
  dominated by:
  - number of inputs (2^n if naive)